ICT261 Java Lab - Unit 4: Exception Handling and Error Management

Student name: Isabel Musenje Student number: 202500362

Files submitted with this document: MarksImporter.java (includes DuplicateStudentException) and marks.txt

Activity A: Predict and explain

Statement	Exception	Type	Prevention / handling
Integer.parseInt("ABC");	NumberFormatException	Unchecked	Catch NumberFormatException around the parse, or validate that the text is numeric first, and report the bad value.
String name = null; name.length();	NullPointerException	Unchecked	Check `name != null` before calling methods on it, or avoid assigning null.
new java.io.FileReader("missing.txt");	FileNotFoundException	Checked	Catch it in try/catch (or declare throws), and check the file exists / the name is right before opening.

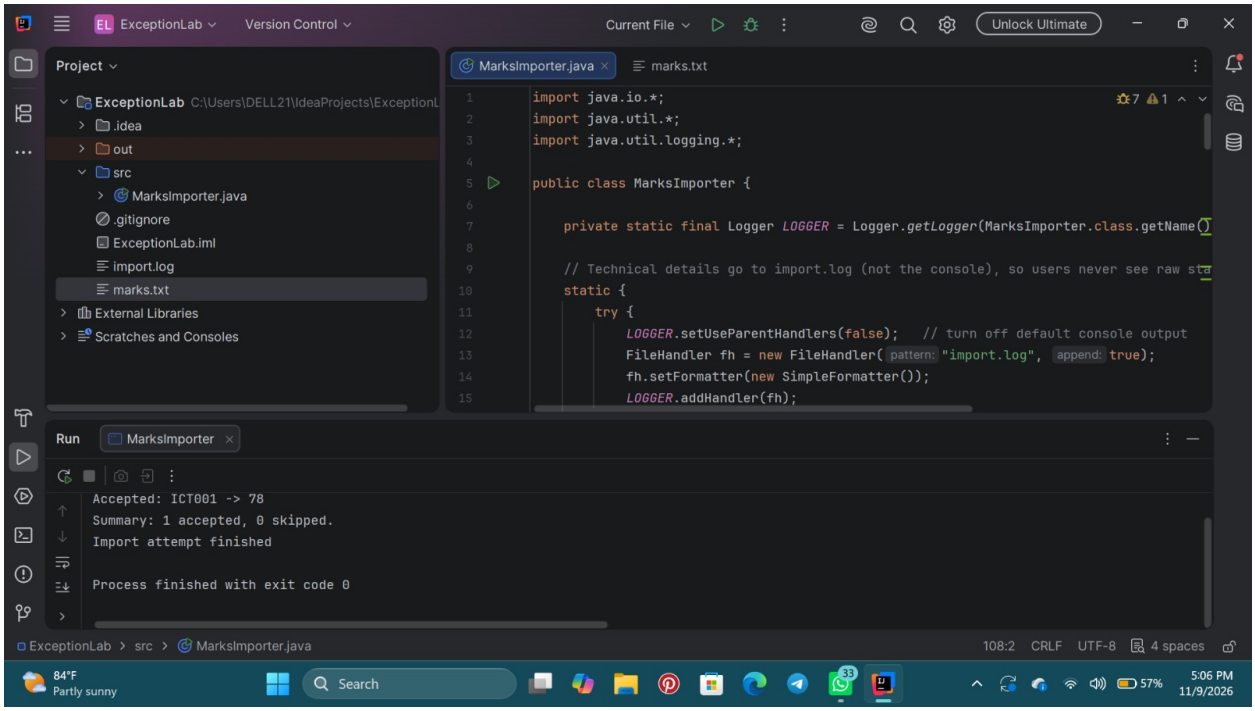
Why OutOfMemoryError should not normally be caught to continue: It is an Error, not an Exception, and it means the JVM has run out of memory and may be in an unstable state. Catching it rarely fixes the cause, so the program may fail again immediately or continue with corrupted data; it is better to let it stop and fix the memory problem.

Activities B and C: Decision to skip or stop on an invalid record

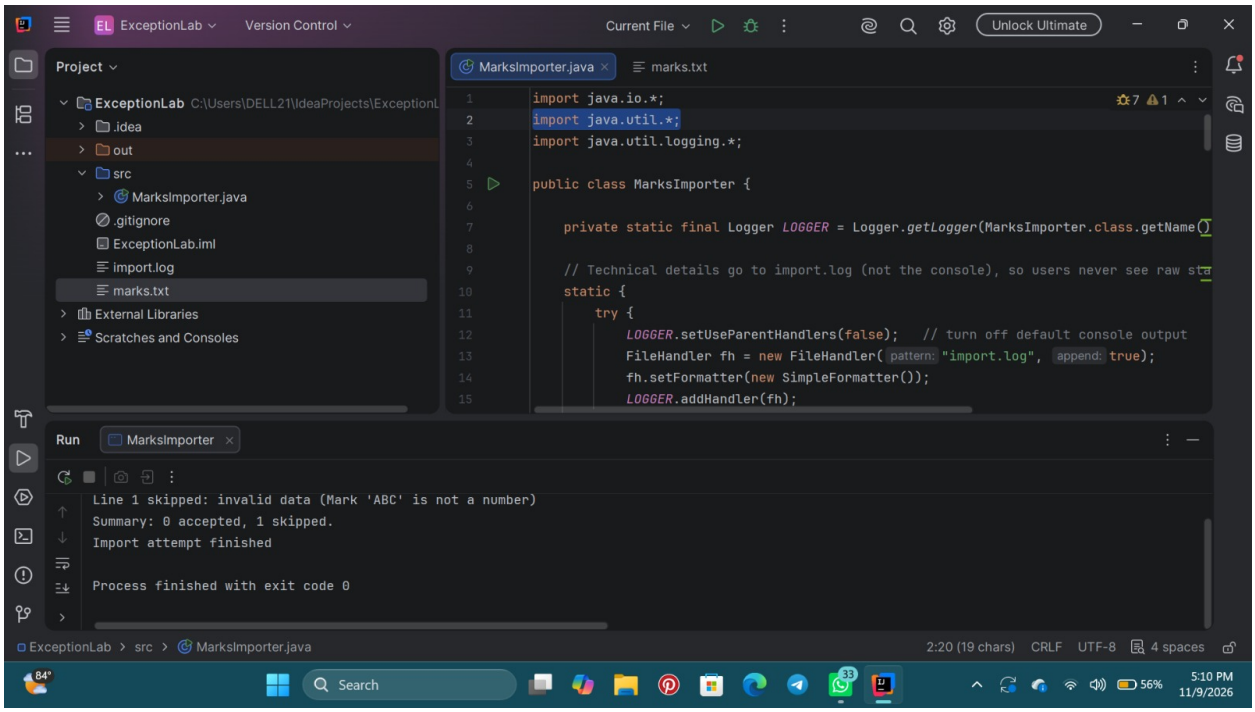
I decided that an invalid record is **skipped**, not allowed to stop the whole import. Each bad line (wrong number of fields, non-numeric mark, mark outside 0..100, or a duplicate student number) is caught inside the loop, reported to the user with its line number in a friendly message, and recorded in import.log with the exception type and technical cause. The importer then carries on with the next line, so one faulty record cannot block the valid ones, and a summary of accepted and skipped records is printed at the end. A missing or unreadable file is different: there is nothing to import, so the method throws IOException to main(), which shows a friendly message, logs the details, and the finally block prints "Import attempt finished".

Test results (screenshots)

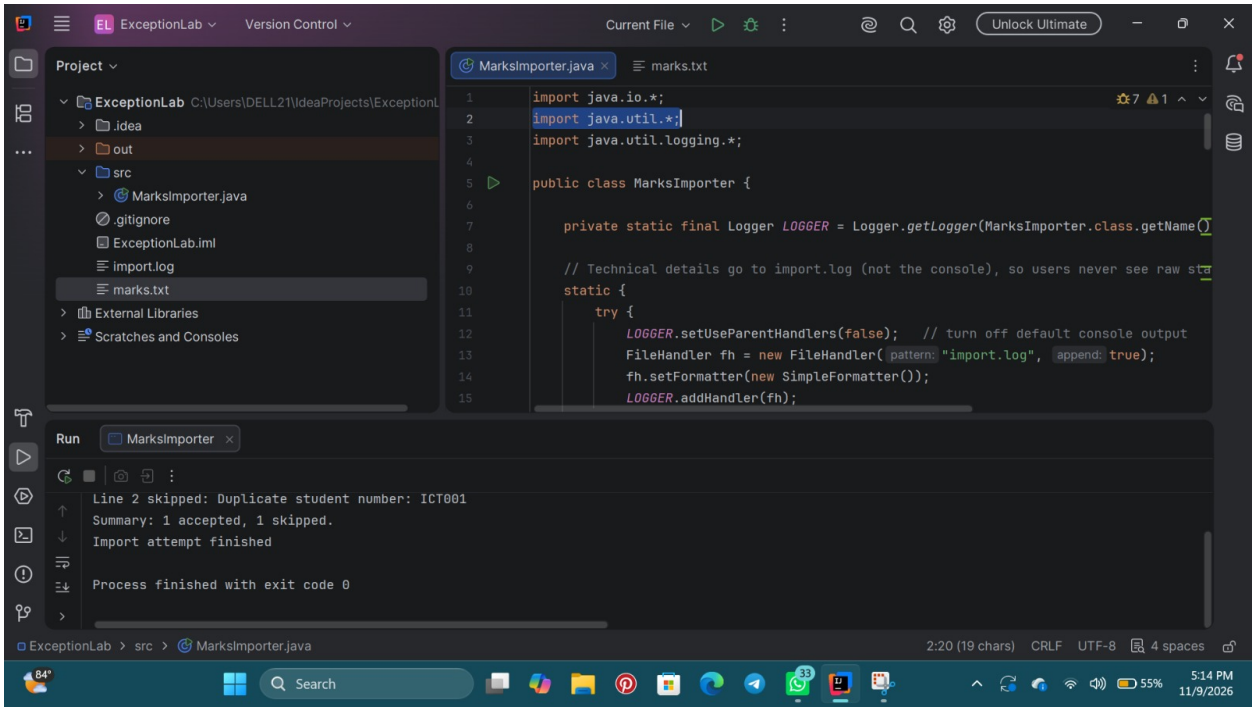
T1 - Input: marks.txt contains ICT001,78. Expected: Valid mark accepted. Done: [x]



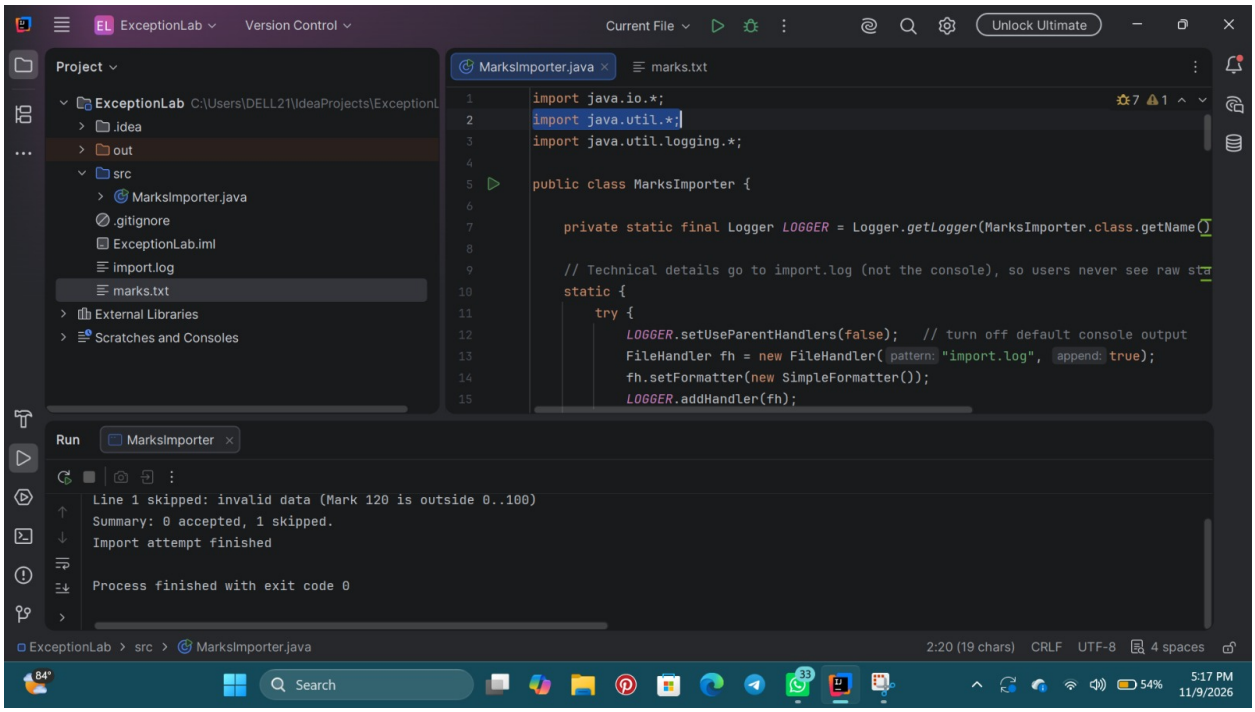
T2 - Input: ICT002,ABC. Expected: Invalid numeric value reported safely. Done: [x]



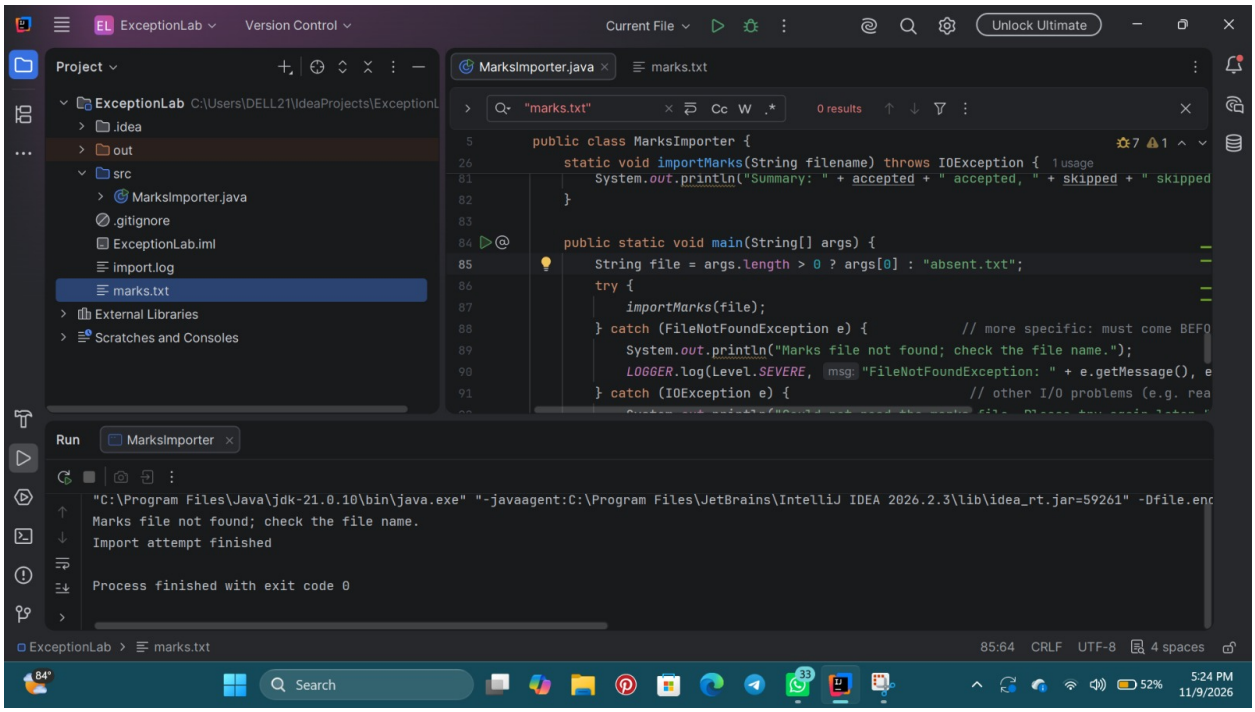
T3 - Input: ICT001,65 after ICT001,78. Expected: Duplicate student ID identified. Done: [x]



T4 - Input: ICT004,120. Expected: Out-of-range mark rejected. Done: [x]



T5 - Input: Filename changed to absent.txt. Expected: Friendly file-not-found message; finally runs. Done: [x]



Activity D: Engineer's review

1. Which of your exceptions is checked, and where does throws appear? Why?

IOException (and its subclass FileNotFoundException) is checked, so the compiler forces me to handle or declare it. `throws IOException` appears on the importMarks method signature because file reading can fail and importMarks does not handle that itself; it passes it up to main(), which shows the friendly message. My custom DuplicateStudentException is unchecked because it extends RuntimeException, so callers are not forced to catch it.

2. Difference between throw and throws

`throw` is a statement that actually raises an exception at that moment, for example `throw new DuplicateStudentException(id);`. `throws` is part of a method signature that warns callers the method may pass a checked exception up, for example `static void importMarks(String filename) throws IOException`.

3. Why a friendly message for the user but technical details in the logger?

Ordinary users cannot act on stack traces and class names, and raw traces can confuse them or expose internal details of the system. A short friendly message tells them what to do (for example, check the file name), while the logger keeps the exception type and cause so a developer can diagnose and fix the problem later.

4. What would go wrong with an empty catch block, or with catching Exception before FileNotFoundException?

An empty catch block swallows the error: the program carries on as if nothing happened, the user is never told, nothing is logged, and the bug becomes very hard to find. Catching Exception before FileNotFoundException makes the specific catch unreachable, because Exception already covers it; Java gives a compile error. Specific exceptions must be caught first, then more general ones.